Brinkman Penalization in Basilisk

Physical basis, mathematical formulation, and algorithmic translation of the uploaded implementations

Scope. This report explains the two supplied Basilisk codes — `binkmanSimple.c` and `binkmanViscosity.c` — together with the modified headers `centeredBinkman.h` and `viscosityBinkman.h`, and relates them to the coupled VOF–Brinkman methodology described in Sharaborin, Rogozin and Kasimov (*Fluids*, 2021).

Prepared from the uploaded source files and paper

1. Problem statement and modeling philosophy

Both supplied programs model a rigid disk moving inside a viscous liquid without using a body-fitted mesh. Instead of cutting the mesh to match the solid boundary, they extend the Navier–Stokes equations to the whole Eulerian domain and impose the rigid-body motion through a penalization term. Physically, the solid is treated as a region of vanishing permeability; mathematically, the fluid velocity is relaxed toward the prescribed solid velocity inside the mask defined by the solid volume fraction χ.

$$\nabla \cdot \boldsymbol{u} = 0, \tag{1}$$

$$\rho \left(\frac{\partial \boldsymbol{u}}{\partial t} + \nabla \cdot (\boldsymbol{u} \otimes \boldsymbol{u}) \right) = -\nabla p + \nabla \cdot \left[\mu \left(\nabla \boldsymbol{u} + \nabla \boldsymbol{u}^T \right) \right] + \boldsymbol{f}_B + \rho\, \boldsymbol{a}_{\text{ext}}, \tag{2}$$

$$\boldsymbol{f}_B = -\frac{\rho \chi}{\eta} (\boldsymbol{u} - \boldsymbol{U}_s). \tag{3}$$

Here χ is the solid mask, equal to 1 in the solid, 0 in the pure fluid, and intermediate in cut cells. The quantity η has dimensions of time and plays the role of permeability divided by viscosity in Brinkman-type reasoning. When η is small, the relaxation is stiff and the velocity inside the solid quickly approaches the rigid velocity $\boldsymbol{U}_s$. The paper emphasizes that the thickness of the Brinkman layer scales like $\delta \sim \sqrt{\eta \nu}$, so η should be tied to the grid size if the layer is to be neither grossly under-resolved nor unnecessarily thick.

$$\eta \approx \frac{(mh)^2}{\nu}. \tag{4}$$

This scaling is the main bridge between the continuous penalization model and the discrete algorithm. In other words, η is not just a physical parameter; it is a numerical closure that controls how the no-slip condition is realized on the mesh.

2. What the uploaded paper contributes

The uploaded paper formulates an algorithm for incompressible multiphase flow with two fluid phases handled by VOF and solids handled by Brinkman penalization. Its central governing equations are the incompressible balance, a one-field momentum equation for the whole domain,

a VOF transport equation for the fluid fraction, and a Brinkman source term that enforces the solid velocity. The paper also explains why one often chooses the solid viscosity so that the kinematic viscosity is continuous across the fluid-solid interface, and why the face velocity may require correction near solids to reduce penetration of one phase into another.

The key point for your codes is that the paper treats the viscous term and the Brinkman term implicitly in the same step. That is the philosophy implemented in `binkmanViscosity.c` together with `centeredBinkman.h` and `viscosityBinkman.h`. The simpler code, `binkmanSimple.c`, keeps the Basilisk Navier–Stokes solver unchanged and adds the Brinkman force explicitly as a body acceleration. This is not identical to the paper algorithm, but it is a legitimate and much simpler variant.

Table 1: Side-by-side view of the two Brinkman implementations

Aspect	**`binkmanSimple.c`**	**`binkmanViscosity.c`**	**Consequence**
How the penalization enters the momentum equation	As an explicit face-centered acceleration in the `acceleration` event.	Inside the implicit viscous multigrid solve through an added diagonal term $K_b = \chi\,\Delta t/\eta$.	Simple version is easy to read and modify; implicit version is tighter but more intrusive.
Material properties	Uniform fluid properties; the solid shares the same density and kinematic viscosity.	Cell-wise mixtures for density and viscosity are built from χ, with face fields α and μ updated every step.	The implicit version is closer to the multiphase formulation in the paper.
Penalization coefficient	Uniform η based on the finest-grid spacing chosen through `maxlevel`.	Local $\eta = (m_{\text{brink}}\,\Delta)^2/\nu_f$, updated from the current cell size.	The second version follows the layer-resolution logic of the paper more directly.
Hydrodynamic force used for disk dynamics	Integrated from $\chi/\eta \cdot (\boldsymbol{u} - \boldsymbol{U}_s)$.	Integrated from $\rho\chi/\eta \cdot (\boldsymbol{u} - \boldsymbol{U}_s)$.	Difference matters when density varies; here densities are equal so the distinction is mostly methodological.
AMR	Enabled by default on χ and $\boldsymbol{u}$.	Disabled by default in the supplied file.	This affects cost and interface sharpness.
Solver footprint	Uses Basilisk standard centered solver unchanged.	Requires `centeredBinkman.h` and `viscosityBinkman.h`.	The simple version is the cleaner pedagogical entry point.

3. Geometry, unknowns, and solid kinematics in the supplied scripts

Both programs solve a two-dimensional channel problem of size $H \times H$ with a disk of radius R_c. The disk center (x_c, y_c) and translational velocity (U_{cx}, U_{cy}) are evolved in time. Rotation is deliberately omitted, so the solid velocity field is spatially uniform inside the disk. The disk geometry is reconstructed each time step with Basilisk `fraction()` applied to the level-set function $R_c^2 - (x - x_c)^2 - (y - y_c)^2$.

$$\chi(x,y) = \text{fraction}\Big[R_c^2 - (x-x_c)^2 - (y-y_c)^2\Big]. \tag{5}$$

The rigid-body update itself is explicit and separate from the fluid solve. The codes integrate the reaction force generated by the penalization term, then update the centroid position and velocity by forward Euler. This means the fluid solver computes the Eulerian momentum exchange, while the particle motion is a lightweight Lagrangian post-processing step.

$$\boldsymbol{F}_h = \int_\Omega \frac{\chi}{\eta}(\boldsymbol{u} - \boldsymbol{U}_s)\, dV \qquad \text{(simple code)}, \tag{6}$$

$$\boldsymbol{F}_h = \int_\Omega \frac{\rho\chi}{\eta}(\boldsymbol{u} - \boldsymbol{U}_s)\, dV \qquad \text{(implicit-viscosity code)}, \tag{7}$$

$$\boldsymbol{U}_c^{n+1} = \boldsymbol{U}_c^n + \Delta t\, \frac{\boldsymbol{F}_h}{m_p}, \qquad \boldsymbol{x}_c^{n+1} = \boldsymbol{x}_c^n + \Delta t\, \boldsymbol{U}_c^n. \tag{8}$$

4. The simple implementation: explicit Brinkman forcing in `binkmanSimple.c`

The simple version is pedagogically clean because it leaves Basilisk standard `centered.h` untouched. The core idea is: build χ, η, and the rigid velocity field $\boldsymbol{U}_s$ in user code; then add the penalization force as an extra face acceleration $\boldsymbol{a}$ during the `acceleration` event. Since Basilisk already knows how to incorporate a face-centered body force into its projection method, nothing else in the solver needs to change.

$$\boldsymbol{a}_{B,f} = -\frac{\chi_f}{\eta_f}(\boldsymbol{u}_f - \boldsymbol{U}_{s,f}). \tag{9}$$

In the file, χ is cell-centered but the forcing is applied on faces, so the code uses simple arithmetic averaging from neighboring cells. This is physically consistent with Basilisk's staggered layout: the pressure correction and body forces live naturally on faces, whereas the primary velocity unknowns live at cell centers. The parameter η is chosen once from the finest mesh spacing estimated with `maxlevel`, so the penalization stiffness is uniform across the solid even if AMR changes the local cell size.

This design has three consequences. First, the implementation is compact and transparent. Second, the penalization is treated explicitly, so the time step must remain small enough that the forcing does not create stiffness problems. Third, the underlying viscous solve remains exactly the standard Basilisk solve, which makes the code easier to debug and compare against a reference run without Brinkman terms.

- The event `properties()` sets $\mu_f = \nu_f$ on faces and refreshes χ, η, and $\boldsymbol{U}_s$.
- The event `acceleration()` writes the penalization body force directly into the face vector $\boldsymbol{a}$.
- The event `end_timestep()` integrates the reaction force and updates the disk state.
- The event `adapt()` refines on both χ and $\boldsymbol{u}$, so the disk boundary and the wake receive resolution.

5. The more sophisticated implementation: implicit coupling in `binkmanViscosity.c`

The second version attempts to follow the spirit of the paper more closely. Instead of appending the penalization as an explicit body force, it inserts it directly into the implicit viscous solve. The modified viscous subproblem is

$$\frac{\rho}{\Delta t}(\boldsymbol{u}^{n+1} - \boldsymbol{u}^*) = \nabla \cdot \left[\mu \left(\nabla \boldsymbol{u}^{n+1} + \nabla \boldsymbol{u}^{n+1,T}\right)\right] - \frac{\chi}{\eta}(\boldsymbol{u}^{n+1} - \boldsymbol{U}_s^{n+1}). \tag{10}$$

After discretization, the extra term contributes a positive diagonal coefficient $K_b = \chi \Delta t/\eta$ to the linear system. That is a strong numerical mechanism: inside the solid, K_b is large, so the linear solve itself drives the unknown velocity toward $\boldsymbol{U}_s$. The no-slip constraint is therefore enforced at the same stage where viscous diffusion is handled. This is why the method is tighter and more faithful to the stiff character of the penalization model.

$$K_b = \frac{\chi \Delta t}{\eta}. \tag{11}$$

The file `update_material_fields()` reconstructs χ and then mixes density and viscosity through χ. In your particular run setup $\rho_s = \rho_f$ and $\nu_s = \nu_f$, so these fields do not create strong jumps, but the machinery is present for a more general extension. Unlike the simple code, η is local and uses the current cell size Δ, namely $\eta = (m_{\text{brink}}\Delta)^2/\nu_f$. This aligns well with the paper's recommendation that the Brinkman layer thickness should track the mesh spacing.

Conceptually, this version is closer to a monolithic treatment of the stiff source. Numerically, however, it is also more delicate: one has modified the linear operator, the multigrid residual, the relaxation step, and the data that must be restricted across levels. That is why it is more sophisticated, but also why it is harder to trust without careful verification.

6. What `centeredBinkman.h` changes in the Navier–Stokes solver

The header `centeredBinkman.h` is intentionally conservative. It remains a centered, projection-based incompressible solver using the standard Basilisk event sequence. The large conceptual structure is unchanged:

1. choose Δt from CFL and event constraints,
2. update material properties,
3. predict advecting face velocities,
4. advance advection,
5. solve the implicit viscous step,
6. add explicit accelerations,
7. project to obtain a divergence-free face velocity.

What changes is the hook used in the viscous stage. Instead of calling the standard viscosity solver, this header calls the modified Brinkman-aware viscosity routine and stores a temporary field often denoted `ubrinkman`. In effect, the header is the place where Basilisk's operator

splitting is told that the viscous stage is no longer pure diffusion: it is diffusion plus rigid-velocity relaxation inside the solid.

This is an important physical point. Basilisk is not a monolithic black-box Navier–Stokes code. It is an event-driven time integrator in which each event corresponds to a mathematically distinct substep. Therefore, modifying one header file can change the meaning of one entire operator in the PDE splitting. That is exactly what happens here.

The staggered placement of variables remains standard Basilisk practice. Cell-centered velocities are the main unknowns. Face-centered velocities are the transport and projection variables. Pressure and pressure gradients appear through the projection stage. Face-centered storage is especially important because the Poisson projection enforces incompressibility on fluxes, and fluxes live naturally on faces.

7. What `viscosityBinkman.h` changes in the linear algebra

The real mathematical modification is in `viscosityBinkman.h`. The standard viscous implicit solve in Basilisk is a vector elliptic problem. The Brinkman term turns it into a vector Helmholtz problem with a spatially varying diagonal shift. The code introduces three global fields — `bpm_chi`, `bpm_eta`, and `bpm_us` — and passes them into the multigrid structure together with μ, ρ, and Δt.

$$L(\boldsymbol{u}) = \text{viscous diffusion} - (\lambda + K_b)\boldsymbol{u} + K_b\boldsymbol{U}_s. \tag{12}$$

Two places are crucial. In `residual_viscosity()`, the code computes the residual of the discrete operator with the added K_b terms. In `relax_viscosity()`, the smoother updates each velocity component using the modified denominator (the local diagonal) and a right-hand side that now includes $K_b\boldsymbol{U}_s$. In plain language: the linear solver does not just diffuse momentum; it also relaxes the local fluid state toward the rigid velocity wherever χ is nonzero.

Because the operator is solved by multigrid, χ, η, and $\boldsymbol{U}_s$ must be restricted to all coarser levels before the solve. That is why `viscosity()` explicitly restricts not only μ and ρ but also the Brinkman fields. Without this, coarse-grid corrections would be inconsistent with the fine-grid penalization operator.

8. The Basilisk Navier–Stokes algorithm in physical terms

Basilisk's centered solver is best understood as an operator-splitting projection method on a staggered adaptive grid. Instead of advancing all terms at once, it advances them in a sequence tailored to the structure of the equations. The physical momentum equation is broken into convection, viscous diffusion, explicit body forces, and the pressure constraint. The pressure is not evolved by its own constitutive equation; it is computed as the Lagrange multiplier that enforces incompressibility.

$$\boldsymbol{u}_t + N(\boldsymbol{u}) = -\alpha\nabla p + V(\boldsymbol{u}) + \boldsymbol{a}, \qquad \nabla\cdot\boldsymbol{u} = 0, \qquad \alpha = 1/\rho. \tag{13}$$

The centered velocity $\boldsymbol{u}$ is the main state variable. A provisional face velocity is first predicted for fluxing the momentum. A projection then removes the divergent part of this provisional velocity. After that, viscous effects are added implicitly. Finally, body-force and pressure corrections are assembled into the field $\boldsymbol{g}$, and a second projection yields a divergence-free face velocity for the next step.

Table 2: How Basilisk translates the continuous equations into an event-driven algorithm

Stage	Continuous operation	Implementation idea in `centeredBinkman.h` / Basilisk
1	Choose Δt from CFL and pending events.	`stability` sets `dt = dtnext(timestep(uf, dtmax));` the face velocity controls stability.
2	Update fluid properties at $t + \Delta t/2$.	`properties` rebuilds μ, α, ρ and, in the Brinkman case, χ, η, $\boldsymbol{U}_s$.
3	Predict a face velocity at half step.	`prediction()` uses a Bell–Colella–Glaz extrapolation from centered $\boldsymbol{u}$ and the old pressure-gradient/acceleration field $\boldsymbol{g}$.
4	Advance the convective term.	`advection_term` projects the predicted face velocity and then advects the centered velocity components with `uf`.
5	Advance viscous and Brinkman terms.	`viscous_term` temporarily adds $\Delta t\,\boldsymbol{g}$, solves the implicit vector Helmholtz problem, stores `ubrinkman`, then removes $\Delta t\,\boldsymbol{g}$.
6	Assemble explicit body-force terms.	`acceleration` fills the face field $\boldsymbol{a}$, then builds `uf = face_value(u) + dt*a`.
7	Enforce incompressibility.	`projection` solves $\nabla \cdot (\alpha \nabla p) = \nabla \cdot \boldsymbol{u}_f^*/\Delta t$ and corrects both `uf` and the centered velocity through $\boldsymbol{g}$.
8	Move to next step / adapt grid.	`end_timestep` allows user-defined updates; `adapt` refines and then `properties` is called again on trees.

9. From equations to code: detailed interpretation of each event

The event-based organization is one of Basilisk's most useful conceptual features. It is not merely programming style; it mirrors the operator splitting. The event `stability` chooses a time step from the current face velocity `uf`. The event `properties` refreshes physical coefficients, which in your case also means rebuilding the solid mask and the Brinkman parameters. The event `advection_term` predicts a face velocity at half step and uses it in a Bell–Colella–Glaz update of the centered velocity field. The event `viscous_term` then solves the stiff diffusion problem implicitly. The event `acceleration` assembles body forces on faces. The event `projection` solves the Poisson problem for pressure and projects the face velocity. Any user-defined event `end_timestep` can then consume the updated fields, which is exactly what your particle-dynamics update does.

This design explains how the physical equations become an actual algorithm. Continuous fields are stored either at cell centers or on faces. Differential operators become finite-volume balances. The pressure gradient is represented on faces, because that is where flux corrections must be applied to make face fluxes divergence-free. Convective transport uses upwind-biased reconstruction through BCG. Diffusion and Brinkman relaxation are solved through multigrid because they would be too stiff to advance explicitly at competitive time steps. Mesh adaptation is executed afterward, and the property fields are refreshed to stay consistent with the new quadtree layout.

10. Why the simple code can work well despite being less elaborate

The simple code is less faithful to the paper's fully implicit philosophy, but it has an important practical virtue: it minimizes the number of moving parts. All pressure-projection and viscous machinery are the standard Basilisk pieces, which are already robust. The only new ingredient

is the extra face force. For moderate time steps and equal material properties, this can produce very good results. In fact, the `README` explicitly notes that the author found the simple version closer to the COMSOL comparison. That is plausible: fewer modifications can mean fewer opportunities for operator inconsistency or multigrid degradation.

From a numerical-analysis viewpoint, one can say that the simple version sacrifices some formal tightness in exchange for implementation robustness. The sophisticated version is closer to the target PDE splitting in the paper, but only if the modified multigrid operator, boundary treatments, restriction, and coefficient updates are all perfectly consistent. This trade-off is common in immersed methods.

11. Main differences between the supplied implementations and the paper

- The paper addresses coupled multiphase VOF–Brinkman flow; your example problems are effectively single-fluid disk-entrainment tests.
- The paper uses a face-velocity correction near solids to limit fluid penetration; the supplied disk scripts do not implement the nonlinear VOF–solid correction logic because there is no second fluid interface to protect.
- `binkmanSimple.c` uses a uniform η based on the finest grid, whereas the paper recommends η tied to the local mesh spacing through $\eta \approx (mh)^2/\nu$.
- `binkmanViscosity.c` follows the paper more closely by embedding Brinkman terms into the implicit stage and by reconstructing mixed material fields.
- Both scripts use a simple forward-Euler rigid-body update; the paper itself focuses mainly on prescribed solid motion rather than fully coupled rigid-body dynamics.

12. Practical reading guide for your files

- Read `binkmanSimple.c` first if your goal is conceptual clarity.
- Read `binkmanViscosity.c` next to see how χ, η, ρ, μ, α, and $\boldsymbol{U}_s$ are prepared for the implicit operator.
- Then read `centeredBinkman.h` as a map of the global time integrator: it tells you where each physical process enters.
- Finally read `viscosityBinkman.h` line by line, because that is where the PDE is transformed into the actual linear system solved by multigrid.

13. Conclusions

The two uploaded implementations realize the same physical idea — rigid-body enforcement through Brinkman penalization — but at two different levels of numerical coupling. The simple code appends the penalization as an explicit body force and therefore leverages Basilisk's standard incompressible solver almost unchanged. The advanced code inserts the same physics into the implicit viscous solve by modifying the multigrid operator with a local Helmholtz-like diagonal $K_b = \chi\Delta t/\eta$. The first route is easier to understand and often easier to make robust. The

second route is mathematically closer to the coupled VOF–Brinkman algorithm of the paper and better suited to stiff penalization, variable properties, and general multiphase extensions.

At the level of Basilisk itself, the central lesson is that the Navier–Stokes equations are not advanced as one opaque black box. They are split into advection, diffusion, body forcing, and projection subproblems on a staggered adaptive finite-volume mesh. Understanding where a term enters this sequence is the key to understanding both the code and the physics. In your files, the decisive question is therefore not simply what the Brinkman term is, but where it is inserted in the event sequence and which discrete operator is responsible for enforcing it.

14. References used for this report

1. Sharaborin, E. L., Rogozin, O. A., and Kasimov, A. R. (2021). *The Coupled Volume of Fluid and Brinkman Penalization Methods for Simulation of Incompressible Multiphase Flows. Fluids*, 6, 334.
2. Uploaded source file: `binkmanSimple.c`.
3. Uploaded source file: `binkmanViscosity.c`.
4. Uploaded source file: `centeredBinkman.h`.
5. Uploaded source file: `viscosityBinkman.h`.
6. Uploaded project note: `README.md`.